StatCore Multilayer Perceptrons: NumPy Backpropagation, Weight-Decay Selection, Reliability Heads, and Predictive Uncertainty

Jabulani Monchwe
Applied Business Analytics and Decision Making
North-West University
IDEA-Research Club Chairperson
Johannesburg, South Africa
monchwej.25@gmail.com

Martin Chanza
Applied Business Analytics and Decision Making
North-West University
Mafikeng, South Africa
martin.chanza@nwu.ac.za

Olorato Motitswane
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
oloratomotitswane@gmail.com

Mfundo Monchwe
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
diditmfundo@gmail.com

***Abstract*—We give a source-faithful account of StatCore's NumPy feed-forward classifier and regressor, including the `base`, `stat_regularised`, `reliability_head`, and `uncertainty_head` variants. The paper derives the implemented ReLU forward pass, softmax cross-entropy and squared-error/Gaussian losses, all-layer reverse-mode gradients, the exact AdamW-style parameter update, validation early stopping, held-out logistic reliability model, and Monte Carlo dropout summaries. Particular care is taken to distinguish the classifier's predictive entropy from calibrated correctness probability, and the regressor's heteroscedastic output from its total predictive standard deviation. Empirical comparisons use scikit-learn only as the conventional reference MLP and dataset provider, and compare all four NumPy variants on three fixed-seed splits of nonlinear/noisy data and bundled Breast Cancer Wisconsin and Diabetes data. The reported measurements do not establish that a variant improves on the standard reference; uncertainty and reliability quality require dedicated evaluation beyond predictive accuracy.**

***Index Terms*—neural networks, backpropagation, AdamW, early stopping, calibration, Monte Carlo dropout, heteroscedastic regression**

I. Introduction

A multilayer perceptron (MLP) represents a nonlinear map by affine layers and elementwise activations, trained by backpropagation and stochastic optimization [1], [2]. ReliableML's StatCore implementation provides an independent NumPy version with fixed or validation-selected weight decay and two reliability-oriented branches. This paper formalizes those exact source behaviors, not an idealized textbook MLP. For example, every weight matrix including the output layer receives He-scaled Gaussian initialization, biases are not decayed, and the reliability head is fitted to one held-out subset rather than by nested cross-fitting.

The exact implementation is `src/ml_reliability/statcore/mlp/mlp.py`; shared helpers are in `statcore/_common.py`, and the library contract in `statcore/SPEC.md`. All StatCore networks and heads use NumPy and the package's own logistic estimator. Scikit-learn is used only for the standard experimental comparator and bundled data access.

II. Problem Setup and Preprocessing

Let $X \in \mathbb{R}^{n \times d}$ with finite values. The implementation requires at least six rows, at least one feature, and a one-dimensional response (an $(n, 1)$ target is flattened). Classification stores the sorted unique labels $\mathcal{C} = (c_1, \ldots, c_C)$ and one-hot target matrix $T \in \{0, 1\}^{n \times C}$, rejecting fewer than two classes. Regression checks finite numeric y, stores $\bar{y} = n^{-1} \sum_i y_i$ and population scale $s_y = \mathrm{std}(y)$ (substituting 1 if this is exactly zero), and trains on $T_i = (y_i - \bar{y})/s_y$. Outputs are transformed back to target units for prediction.

The input scaler computes a population mean and standard deviation over all rows passed to `fit`, replacing each scale $\leq 10^{-12}$ with 1. Importantly, this occurs before the training/validation/head partition. It is unsupervised, but the validation/head feature values affect the training scaler; it is not a fully fold-isolated preprocessing pipeline. Finite numeric arrays, consistent feature count, and a fitted model are required at inference.

Let layer widths be $d_0 = d, d_1, \ldots, d_L$, where L is the number of affine layers and $(d_1, \ldots, d_{L-1}) =$ `hidden_layer_sizes`. The default hidden widths are $(32, 32)$; the constructor rejects an empty hidden list or widths below one after integer conversion. With seeded generator, each element of each matrix is initialized independently as

$$W_{ab}^{(\ell)} \sim \mathcal{N}\left(0, \frac{2}{d_{\ell-1}}\right), \qquad b^{(\ell)} = 0, \tag{1}$$

including the output matrix. Equation (1) is the code's rule; it does not switch to a separate softmax/logit or variance-head initializer.

III. Forward model, objectives, and gradients

A. Hidden-layer forward pass and inverted dropout

For row-batch $A^{(0)} = X$, each hidden preactivation and ReLU output are

$$Z^{(\ell)} = A^{(\ell-1)} W^{(\ell)} + \mathbf{1}(b^{(\ell)})^\top, \tag{2}$$

$$H^{(\ell)} = \max(Z^{(\ell)}, 0), \quad \ell = 1, \ldots, L-1. \tag{3}$$

Only the `uncertainty_head` variant applies dropout, and only during training and stochastic inference. For dropout rate $p =$ `dropout`, each hidden layer receives an independent mask $M_{ij}^{(\ell)} = B_{ij}^{(\ell)}/(1-p)$ with $B \sim \text{Bernoulli}(1-p)$ and activation $A^{(\ell)} = H^{(\ell)} \odot M^{(\ell)}$. All other variants use $p_{\text{drop}} = 0$ even when the constructor's `dropout` is nonzero. At deterministic inference for non-uncertainty variants, $A^{(\ell)} = H^{(\ell)}$. The linear output is

$$O = A^{(L-1)} W^{(L)} + \mathbf{1}(b^{(L)})^\top. \tag{4}$$

B. Classification cross-entropy

For logits $o_i \in \mathbb{R}^C$, the implementation uses the numerically shifted softmax $p_{ic} = \exp(o_{ic} - m_i)/\sum_r \exp(o_{ir} - m_i)$, $m_i = \max_c o_{ic}$. The batch objective and output derivative are

$$\mathcal{L}_{\text{cls}} = -\frac{1}{B} \sum_{i=1}^{B} \sum_{c=1}^{C} t_{ic} \log(\max(p_{ic}, 10^{-12})), \tag{5}$$

$$\frac{\partial \mathcal{L}_{\text{cls}}}{\partial o_{ic}} = \frac{p_{ic} - t_{ic}}{B}. \tag{6}$$

No class weighting or explicit label smoothing is used. Training probabilities are not passed through a separate temperature or calibration layer.

C. Squared-error regression

For base and `stat_regularised` regression, O has one column and $r_i = o_i - T_i$. The implemented objective is half the mean squared error,

$$\mathcal{L}_{\text{mse}} = \frac{1}{2B} \sum_i r_i^2, \qquad \frac{\partial \mathcal{L}_{\text{mse}}}{\partial o_i} = \frac{r_i}{B}. \tag{7}$$

The factor one-half is retained in the objective and cancels in the reported unscaled regression metrics.

D. Heteroscedastic Gaussian-style regression objective

For the `uncertainty_head` regressor, output dimension is two: $(\mu_i, \ell_i) = (O_{i1}, O_{i2})$, where ℓ is an unconstrained log-variance logit clipped to $[-6, 6]$ at loss evaluation. With $v_i = \exp(\ell_i)$ and $r_i = T_i - \mu_i$, code minimizes

$$\mathcal{L}_{\text{het}} = \frac{1}{2B} \sum_i \left(\ell_i + \frac{r_i^2}{\exp(\ell_i)}\right), \tag{8}$$

$$\frac{\partial \mathcal{L}}{\partial \mu_i} = -\frac{r_i}{B v_i}, \qquad \frac{\partial \mathcal{L}}{\partial \ell_i} = \frac{1 - r_i^2/v_i}{2B}. \tag{9}$$

This is Gaussian negative log likelihood up to the constant $\frac{1}{2}\log(2\pi)$, which is omitted. As implemented, the output is clipped for the forward loss and the analytic derivative in (9) is returned directly; no additional zero-gradient mask is applied when the raw log variance lies outside the clip interval. At prediction, both log-variance and variance calculations use the same $[-6, 6]$ clipping.

E. Backpropagation through all layers

For any of the losses, let $\Delta^{(L)} = \partial \mathcal{L}/\partial O$ from (6), (7), or (9). For layer ℓ the exact matrix gradients are

$$\nabla_{W^{(\ell)}} \mathcal{L} = (A^{(\ell-1)})^\top \Delta^{(\ell)}, \qquad \nabla_{b^{(\ell)}} \mathcal{L} = \sum_{i=1}^{B} \Delta_i^{(\ell)}. \tag{10}$$

For hidden layer $\ell = L-1, \ldots, 1$, propagation is

$$\Delta^{(\ell)} = \left(\Delta^{(\ell+1)} (W^{(\ell+1)})^\top\right) \odot M^{(\ell)} \odot \mathbb{1}[Z^{(\ell)} > 0], \tag{11}$$

where $M^{(\ell)} = 1$ without dropout. The code caches post-dropout activations and masks; its ReLU gate is evaluated as $A^{(\ell)} > 0$. For nonnegative inverted-dropout masks, this is equivalent to the positive pre-dropout ReLU activation gate. ReLU derivatives at exactly zero are set to zero. Applying (10) to each layer yields the complete reverse-mode gradient.

IV. Optimization, validation, and stopping

A. Mini-batch AdamW-style update

Training shuffles row indices with the model's seeded NumPy generator once per epoch and batches them in order. The effective batch size is $B_s = \max(1, \min(\text{int}(\texttt{batch_size}), n_{\text{tr}}))$. For each matrix and bias gradient g_t, moments update as

$$m_t = 0.9 m_{t-1} + 0.1 g_t, \qquad v_t = 0.999 v_{t-1} + 0.001 g_t^2, \tag{12}$$

$$\hat{m}_t = \frac{m_t}{1 - 0.9^t}, \qquad \hat{v}_t = \frac{v_t}{1 - 0.999^t}. \tag{13}$$

The step counter t increments for every minibatch and is shared across layers. For weight matrix $W^{(\ell)}$ and bias $b^{(\ell)}$ the actual update is

$$W_t^{(\ell)} = W_{t-1}^{(\ell)} - \eta \frac{\hat{m}_{W,t}^{(\ell)}}{\sqrt{\hat{v}_{W,t}^{(\ell)}} + 10^{-8}} - \eta \lambda W_{t-1}^{(\ell)}, \tag{14}$$

$$b_t^{(\ell)} = b_{t-1}^{(\ell)} - \eta \frac{\hat{m}_{b,t}^{(\ell)}}{\sqrt{\hat{v}_{b,t}^{(\ell)}} + 10^{-8}}. \tag{15}$$

Here η = `learning_rate` (default 0.01) and λ is the fixed or selected weight-decay coefficient. This is decoupled weight decay on weights only, not an L2 penalty added to the minibatch loss; biases are not decayed. No learning-rate schedule or gradient clipping is implemented. Default `weight_decay` $= 10^{-2}$.

B. Holdout and early stopping

The base, uncertainty, and stat-regularised variants partition the rows passed to fit into a training subset and validation subset using `holdout_split` with seed `random_state+1`; classification partitions stratify by class. For unstratified targets the held-out count is $\max(1, \mathrm{round}(n\,f))$. In the stratified implementation, each class is independently shuffled; for a class of size greater than one the holdout count is rounded and constrained to $[1, n_c - 1]$, while a singleton contributes zero validation rows. `validation_fraction` defaults to 0.15 and must be strictly between zero and 0.9. Validation loss is computed deterministically with dropout off.

After every epoch, if validation loss is less than the best observed value by more than 10^{-5}, the implementation copies all weights and biases and resets the wait counter. Otherwise it increments the counter and stops when `wait >= patience` (default 15). The best saved parameters are restored, not the final epoch parameters. Training is bounded by `max_epochs` (default 150); the initial parameters serve as a fallback best state. There is no validation-set retraining after selection. The call `_train` resets the random generator for each run, so stat-regularised grid candidates share identical initialization and data-order random streams, though their evolving gradients differ.

C. Statistically selected weight decay

For `stat_regularised`, the fixed holdout is shared across all coefficients λ in `wd_grid` (default $(10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1)$). For each grid value the full early-stopping training routine is run from the same random seed. Classification selects the candidate with smallest validation cross-entropy; regression selects smallest validation half-MSE on standardized targets. Scores are stored in `wd_scores_`; the first grid item achieving the minimum is retained because replacement uses a strict comparison. Its best-validation parameters are kept without refitting on training plus validation. The name describes validation-based regularization selection; it does not imply a Bayesian prior, analytic shrinkage, or a statistical risk guarantee. Reusing one validation set to choose among a grid can overfit that set, so external/nested validation is appropriate for performance claims.

V. Reliability head

The `reliability_head` branch first reserves a disjoint head subset of fraction `head_fraction` (default 0.2), using `random_state` and class stratification for classification. The network and its internal early-stopping validation split are trained only on the remainder. For each held-out head row, deterministic network output and final hidden representation h_i yield features

$$f_i = \left[\|h_i\|_2,\ \frac{d^{-1}\|z_i\|_2^2}{\mathrm{mean}_{j,k}(z_{jk}^2) + 10^{-12}},\ \cdots \right]. \tag{16}$$

In the implementation the second component is precisely $\mathrm{mean}_k(z_{ik}^2)/(\texttt{_train_mean_sq} + 10^{-12})$, where `_train_mean_sq` is the mean of every squared standardized training entry. Classification appends the predicted maximum probability, predictive entropy $-\sum_c p_{ic}\log(\max(p_{ic}, 10^{-12}))$, and top-two probability margin. Its binary target is $r_i = \mathbb{1}[\arg\max_c p_{ic} = \arg\max_c t_{ic}]$. Regression appends $|\hat{T}_i|$ and uses $r_i = \mathbb{1}[|\hat{T}_i - T_i| \le \mathrm{median}_{j\in H}\,|\hat{T}_j - T_j|]$; thus the event is relative to the median error on the head subset and is defined in standardized-target units.

If both values of r_i occur, the head fits StatCore `LogisticRegression(alpha=1.0, random_state=...)` to (f_i, r_i); otherwise it stores the constant empirical target value. That logistic estimator standardizes its head features, then fits two-class softmax logits $z_{ic} = w_c^\top \tilde{f}_i + b_c$ by minimizing

$$\mathcal{J}(W, b) = -\frac{1}{|H|}\sum_{i\in H}\log p_{i,r_i} + \frac{1}{2}\sum_{c=0}^{1}\|w_c\|_2^2, \tag{17}$$

$$p_{ic} = \mathrm{softmax}(z_i)_c, \quad z_{ic} = w_c^\top \tilde{f}_i + b_c. \tag{18}$$

with unpenalized intercepts and L-BFGS-B (maximum 300 iterations). Thus the network layers use NumPy, while the optional reliability head delegates to the StatCore logistic implementation, which uses SciPy's L-BFGS-B optimizer. At inference, the same features are recomputed, and `predict_reliability` returns the logistic probability for class 1 or the constant. The model is trained on predictions of a base network that did not see head rows, which avoids direct in-sample outcome targets for those rows. Nevertheless, it has only one small calibration split, its regression event is sample-relative, and no theorem makes its output a calibrated reliability probability under shift. The network is not retrained after the head split. Reliability head is available for either task; it is not a separate neural output neuron.

VI. Uncertainty head and prediction rules

A. Monte Carlo dropout

For `uncertainty_head`, training uses dropout masks in all hidden layers. Inference performs `mc_samples` stochastic forward passes (default 30) from the same fitted weights using a generator initialized with `random_state` on each call. The returned Monte Carlo output array has shape $S \times q \times C$ for classification or $S \times q \times 2$ for regression. Consequently, calls with identical input and fitted parameters are deterministic/reproducible, while the individual draws still vary within a call.

For classification, the prediction is the average of pass-specific probabilities,

$$\bar{p}_{ic} = \frac{1}{S}\sum_{s=1}^{S} \mathrm{softmax}(o_i^{(s)})_c, \tag{19}$$

$$u_i = \frac{-\sum_c \bar{p}_{ic}\log(\max(\bar{p}_{ic}, 10^{-12}))}{\log C}. \tag{20}$$

Thus `predict_uncertainty` returns entropy normalized by $\log C$, not mutual information or a calibrated error probability. `reliability_score` returns $1 - u_i$ for this branch; it can be negative only if numerical issues violate the entropy bound (with valid normalized probabilities, it lies in $[0, 1]$).

For regression, the point estimate is $\bar{\mu}_i = S^{-1}\sum_s \mu_i^{(s)}$ in standardized target units. The uncertainty output in original target units is

$$u_i = s_y\sqrt{\frac{1}{S}\sum_s \exp(\mathrm{clip}(\ell_i^{(s)}, -6, 6)) + \mathrm{Var}_s(\mu_i^{(s)})}. \tag{21}$$

This combines mean predicted aleatoric variance and population variance of MC means (NumPy's default variance, divisor S), then multiplies the standard deviation by s_y. It does not add a Gaussian 2π constant and does not include a separate finite-sample correction. The regressor's `reliability_score` is $[1 + u_i/s_y]^{-1}$; it is a monotone score, not a probability of being correct. Regression prediction transforms $\bar{\mu}$ to $s_y\bar{\mu} + \bar{y}$.

B. Variant summary

VII. Algorithms

A. Network fitting

1) Validate X, y (at least six rows); encode class labels or standardize numeric target. Fit feature standardization on all fit-time X.
2) For `reliability_head`, reserve the seeded head split; otherwise use all rows as the network pool. Split that pool into training and validation indices with seed `random_state+1`.
3) For each fixed or candidate decay value, initialize all layers by (1); initialize Adam moments to zero, best validation loss to infinity, and best state to the initial parameter copy.
4) For each epoch, seeded-shuffle training rows; for each minibatch compute forward activations (and, only for uncertainty, inverted-dropout masks), the task loss and output derivative, then (10)–(11) and update (15).
5) At epoch end evaluate deterministic validation loss. Save parameters only for an improvement exceeding 10^{-5}; otherwise increment wait and stop at patience. Restore best parameters.
6) For `stat_regularised`, retain the first coefficient with lowest validation loss; for `reliability_head`, fit the held-out logistic/constant head after base network fitting.

B. Reliability head and uncertainty prediction

1) For reliability fitting, predict on the reserved head rows; construct final-hidden norm, normalized input energy, and task-specific confidence/error-summary features. Derive correctness or below-median-error labels from this held-out subset.
2) Store a constant when the reliability labels are all identical; otherwise fit the stated StatCore logistic regression. At inference, apply the same feature mapping and return the label-1 probability.
3) For uncertainty inference, initialize the seeded generator and collect $S =$ `mc_samples` network outputs with inference dropout. Average softmax probabilities for classification, or means for regression; compute entropy or (21) respectively.

VIII. Hyperparameters, assumptions, edge cases, and complexity

Constructor defaults are hidden widths $(32, 32)$, $\eta = 0.01$, $\lambda = 0.01$, 150 epochs, batch size 64, patience 15, validation fraction 0.15, dropout 0.1, MC samples 30, head fraction 0.2, decay grid $(10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1)$, and random seed 0. The code validates the variant, nonempty positive integer-converted hidden widths, $0 < f_{\mathrm{val}} < 0.9$, and $0 \le p < 1$. It does not comprehensively range-check learning rate, decay, epoch count, patience, head fraction, MC count, or every grid value. In particular, an invalid or empty decay grid is not proactively rejected. Users should provide positive integer counts, positive finite learning rate and MC count, a nonempty finite nonnegative decay grid, and a sensible head fraction; the implementation's stratified splitter caps per-class holdout counts but does not promise a minimum head size overall.

The architecture assumes finite numeric vectors, a fixed feature set, exchangeable training/validation examples, and a useful Euclidean feature scaling. The softmax probabilities and reliability heads are not inherently calibrated. Validation-based model choice can overfit; the reliability head's empirical event depends on the particular held-out split. MC dropout approximates a stochastic predictive distribution, but its entropy/variance should not be interpreted as calibrated confidence without validation. The Gaussian head assumes a conditional Gaussian likelihood with diagonal per-example scalar variance and can represent nonconstant noise; it does not robustify outliers. Constant regression targets use $s_y = 1$, so standardized targets are zero. Class labels are mapped through sorted unique values; class output order follows that stored list.

Let $P = \sum_{\ell=1}^{L} d_{\ell-1}d_\ell$ be the weight count and $Q = \sum_{\ell=1}^{L} d_\ell$ the biases. For batch size B, a forward/backward pass costs $O(B(P+Q))$ and stores activations $O(B\sum_\ell d_\ell)$, weights, gradients, and four Adam moment arrays of $O(P+Q)$. An epoch processes n_{tr} rows, so E epochs cost $O(En_{\mathrm{tr}}(P+Q))$ up to optimizer constants. Stat-regularised multiplies training cost by the number of decay grid values. A reliability head adds feature extraction and a small logistic fit on the reserved rows. MC prediction costs $O(Sq(P+Q))$ and stores sampled outputs of size $O(SqC)$ for classification

TABLE I
IMPLEMENTED MLP VARIANTS AND THEIR PRECISE BRANCH-LEVEL DIFFERENCES.

Variant	Classifier	Regressor	Additional cost / caution
`base`	Softmax cross-entropy; fixed λ; no dropout	Half-MSE on standardized target; fixed λ	One training run; confidence is not calibrated
`stat_regularised`	Choose λ by validation cross-entropy	Choose λ by standardized validation MSE	One training run per grid value; shared holdout can be overfit
`reliability_head`	Held-out correctness target with confidence, entropy, margin, representation and novelty	Held-out below-median absolute-error target with mean magnitude, representation and novelty	Reserves a head subset; head output is not guaranteed calibrated
`uncertainty_head`	Hidden-layer MC dropout; mean probabilities and normalized entropy	Dropout plus two-output heteroscedastic loss; MC aleatoric+epistemic SD	S inference passes; entropy/SD requires separate empirical validation

or $O(2Sq)$ for regression; this implementation materializes all draws before aggregating.

IX. EMPIRICAL EVALUATION

A. Protocol

Run the script at docs/latex/experiments/mlp_experiment.py from the repository root with `python` in the project virtual environment. The companion utility script defines all generation, splitting, metrics, and reporting. The four StatCore configurations are compared with the standard scikit-learn MLP classifier/regressor, wrapped with scikit-learn standardization. The reference uses hidden widths $(32, 32)$, Adam learning-rate initialization 0.01, $\alpha = 0.01$, batch size 64, at most 100 iterations, early stopping, and patience 12. This is a standard reference configuration, not an assertion of identical optimizer/objective semantics: its regularization and stopping implementation differ from StatCore.

The datasets are 640 generated observations with two nonlinear informative inputs, 18 independent Gaussian irrelevant features, 10% label flips or Gaussian target noise $\sigma = 0.30$, and scikit-learn bundled Breast Cancer Wisconsin (classification) or Diabetes (regression). For seeds 11, 23, and 47, a 75/25 split is made; classification is stratified. Splits are generated in NumPy: the test count is $\lceil 0.25n \rceil$; class counts use proportional floors plus largest fractional remainders with seeded tie-breaking, then within-class and aggregate permutations; regression uses a seeded permutation. The synthetic sample is regenerated for each seed. Each mean and sample SD is over these three test splits. StatCore uses widths $(32, 32)$, learning rate 0.01, fixed decay 0.01, 100 epochs maximum, batch 64, patience 12, validation fraction 0.15, dropout 0.1, 20 MC draws, head fraction 0.2, default decay grid, and the split seed. Each estimator receives one untimed warm-up fit and test prediction on that same training split; reported runtime is the subsequent fit plus test prediction and includes all grid trainings and MC draws as applicable, but excludes data generation and metric calculation.

Classification reports accuracy, natural-log cross-entropy, and equal-width 15-bin ECE computed from predicted class probabilities. Regression reports MSE, MAE, and R^2 in original target units. ECE is $\sum_b (n_b/N)|\mathrm{accuracy}_b - \mathrm{confidence}_b|$ with bins on $[0, 1]$; it is a finite-sample, bin-dependent summary. No test-set hyperparameter selection is performed. The benchmark does not evaluate `predict_reliability` against an independent calibration metric, nor does it test uncertainty coverage or interval calibration; the predictive metrics in the tables cannot validate those heads.

B. Classification results

C. Regression results

The tables retain all measured methods, including losses and runtimes that may favor the standard implementation. The small number of splits, bundled low-dimensional datasets, and single synthetic data-generating mechanism constrain conclusions. Accuracy and ECE do not establish that a reliability score identifies errors, and point prediction metrics do not establish calibrated regression uncertainty. Further work should use locked external data, nested tuning, uncertainty coverage and risk–coverage analysis, subgroup diagnostics, repeated data-generating mechanisms, and confidence intervals over datasets.

X. CONCLUSION

The StatCore MLP combines standard feed-forward learning with four specific implementation variants. The derivations show where standard cross-entropy and MSE gradients coincide with textbook forms and where the actual code adds details such as weight-only decoupled decay, a strict early-stopping threshold, dropout confined to the uncertainty variant, a held-out logistic reliability layer, and clipped heteroscedastic log variance. The experimental protocol is reproducible but intentionally limited; superiority, calibration, and robustness remain empirical questions rather than consequences of the variant names.

TABLE II
CLASSIFICATION RESULTS, MEAN ± SAMPLE SD OVER THREE SEEDS; ELAPSED SECONDS INCLUDE FIT AND PREDICTION.

Dataset	Method	Accuracy	Log loss	15-bin ECE	Time (s)
Synthetic	sklearn standard	0.6333 ± 0.0315	0.9990 ± 0.3149	0.2087 ± 0.1084	0.1164 ± 0.0418
Synthetic	StatCore base	0.5625 ± 0.0062	0.7048 ± 0.0478	0.1214 ± 0.0506	0.0320 ± 0.0145
Synthetic	stat-regularised	0.5542 ± 0.0191	0.6823 ± 0.0248	0.0834 ± 0.0291	0.1974 ± 0.0833
Synthetic	reliability head	0.5625 ± 0.0410	0.7043 ± 0.0386	0.1193 ± 0.0269	0.0341 ± 0.0131
Synthetic	uncertainty head	0.5875 ± 0.0108	0.6771 ± 0.0301	0.0822 ± 0.0278	0.0459 ± 0.0114
Breast cancer	sklearn standard	0.9580 ± 0.0185	0.1132 ± 0.0454	0.0322 ± 0.0161	0.1008 ± 0.0386
Breast cancer	StatCore base	0.9510 ± 0.0242	0.1424 ± 0.0170	0.0349 ± 0.0079	0.0372 ± 0.0230
Breast cancer	stat-regularised	0.9534 ± 0.0283	0.1276 ± 0.0431	0.0342 ± 0.0090	0.1712 ± 0.0540
Breast cancer	reliability head	0.9510 ± 0.0185	0.2167 ± 0.0722	0.0470 ± 0.0110	0.0855 ± 0.0754
Breast cancer	uncertainty head	0.9487 ± 0.0146	0.1196 ± 0.0406	0.0265 ± 0.0053	0.0465 ± 0.0193

TABLE III
REGRESSION RESULTS IN ORIGINAL TARGET UNITS, MEAN ± SAMPLE SD OVER THREE SEEDS.

Dataset	Method	MSE	MAE	R^2	Time (s)
Synthetic	sklearn standard	0.4092 ± 0.0132	0.5187 ± 0.0216	0.4973 ± 0.0503	0.1180 ± 0.0344
Synthetic	StatCore base	0.5035 ± 0.0376	0.5715 ± 0.0147	0.3803 ± 0.0887	0.0929 ± 0.0541
Synthetic	stat-regularised	0.2466 ± 0.0349	0.3977 ± 0.0243	0.6951 ± 0.0656	0.3715 ± 0.0438
Synthetic	reliability head	0.5492 ± 0.0841	0.5933 ± 0.0433	0.3159 ± 0.1917	0.0598 ± 0.0291
Synthetic	uncertainty head	0.3690 ± 0.0471	0.4891 ± 0.0287	0.5464 ± 0.0720	0.1289 ± 0.0424
Diabetes	sklearn standard	3447.3123 ± 303.0937	48.4350 ± 0.6555	0.4076 ± 0.0999	0.2577 ± 0.1055
Diabetes	StatCore base	3767.3793 ± 544.8697	49.6574 ± 1.2546	0.3541 ± 0.1156	0.0429 ± 0.0102
Diabetes	stat-regularised	3559.4609 ± 183.3381	49.6189 ± 0.1811	0.3888 ± 0.0902	0.1906 ± 0.0488
Diabetes	reliability head	3789.8210 ± 674.2387	49.5616 ± 3.1433	0.3456 ± 0.1652	0.0479 ± 0.0067
Diabetes	uncertainty head	3675.8966 ± 212.9897	50.4040 ± 1.6224	0.3726 ± 0.0408	0.0583 ± 0.0132

REFERENCES

[1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, Oct. 1986.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2015, pp. 1026–1034.

[3] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[4] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[5] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proc. 33rd Int. Conf. Machine Learning*, PMLR, vol. 48, 2016, pp. 1050–1059.

[6] A. Kendall and Y. Gal, "What uncertainties do we need in Bayesian deep learning for computer vision?" in *Advances in Neural Information Processing Systems 30*, 2017, pp. 5574–5584.

[7] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Machine Learning*, PMLR, vol. 70, 2017, pp. 1321–1330.

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.